

HDOC DAY-8 ACTIVITY

C PROGRAMMING ASSIGNMENT REPORT

QUESTION 1 – Cube

Write a C program to find the volume, total surface area, and face diagonal of a cube when its side is given.

1. Problem Understanding

- **Input:** Side length of the cube.
- **Output:** Volume, total surface area, and face diagonal.
- **Formula:** Volume = s^3 ; Total Surface Area = $6s^2$; Face Diagonal = $s\sqrt{2}$.

Formula Summary

Quantity	Formula
Volume	$V = s^3$
Total Surface Area	$TSA = 6s^2$
Face Diagonal	$D = s\sqrt{2}$

2. Standard Test Cases

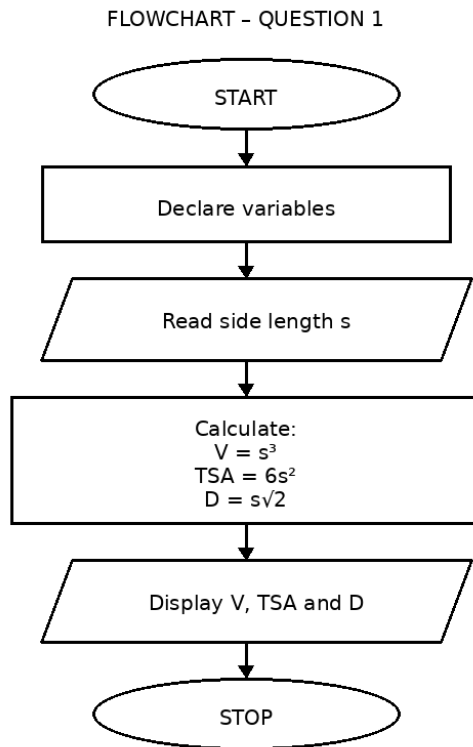
Test Case	Input (s)	Expected Output (s)	Actual Output (s)	Result (Pass/Fail)
1	s = 2 units	Volume = 8.0000 cubic units; TSA = 24.0000 square units; D = 2.8284 units	Volume = 8.0000 cubic units; TSA = 24.0000 square units; D = 2.8284 units	Pass
2	s = 5 units	Volume = 125.0000 cubic units; TSA = 150.0000 square units; D = 7.0711 units	Volume = 125.0000 cubic units; TSA = 150.0000 square units; D = 7.0711 units	Pass
3	s = 10 units	Volume = 1000.0000 cubic units; TSA = 600.0000 square units; D = 14.1421 units	Volume = 1000.0000 cubic units; TSA = 600.0000 square units; D = 14.1421 units	Pass

3. Algorithm

1. Start the program.
2. Declare variables for side, volume, total surface area, and face diagonal.

3. Read the side length of the cube.
4. Calculate volume as $\text{side} \times \text{side} \times \text{side}$.
5. Calculate total surface area as $6 \times \text{side} \times \text{side}$ and face diagonal as $\text{side} \times \sqrt{2}$.
6. Display the volume, total surface area, and face diagonal to four decimal places.
7. Stop the program.

4. Flowchart



5. C Program

```
#include <stdio.h>

#include <math.h>

int main(void)

{

double side, volume, surface_area, face_diagonal;

printf("Enter side length of cube (in units): ");

scanf("%lf", &side);

volume = side * side * side;

surface_area = 6 * side * side;
```

```

face_diagonal = side * sqrt(2);

printf("Volume = %.4f cubic units\\n", volume);

printf("Total Surface Area = %.4f square units\\n", surface_area);

printf("Face Diagonal = %.4f units\\n", face_diagonal);

return 0;

}

```

6. Compilation and Execution Evidence

```

kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)-[~/Desktop]
└─$ gedit question1.c

** (gedit:2947): WARNING **: 14:05:01.527: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:2947): WARNING **: 14:05:01.527: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found

(kali@kali)-[~/Desktop]
└─$ gcc question2.c -o question2
cc1: fatal error: question2.c: No such file or directory
compilation terminated.

(kali@kali)-[~/Desktop]
└─$ gcc question1.c -o question1

(kali@kali)-[~/Desktop]
└─$ ./question1
Enter side length of cube (in units): 2
Volume = 8.0000 cubic units\nTotal Surface Area = 24.0000 square units\nFace Diagonal = 2.8284 units\n

(kali@kali)-[~/Desktop]
└─$ ./question1
Enter side length of cube (in units): 5
Volume = 125.0000 cubic units\nTotal Surface Area = 150.0000 square units\nFace Diagonal = 7.0711 units\n

(kali@kali)-[~/Desktop]
└─$ ./question1
Enter side length of cube (in units): 10
Volume = 1000.0000 cubic units\nTotal Surface Area = 600.0000 square units\nFace Diagonal = 14.1421 units\n

(kali@kali)-[~/Desktop]
└─$

```

7. Verification of All Test Cases

Test Case	Input (s)	Expected Output (s)	Actual Output (s)	Result (Pass/Fail)
1	s = 2 units	Volume = 8.0000 cubic units; TSA = 24.0000 square units; D = 2.8284 units	Volume = 8.0000 cubic units; TSA = 24.0000 square units; D = 2.8284 units	Pass
2	s = 5 units	Volume = 125.0000 cubic units; TSA = 150.0000 square units; D = 7.0711 units	Volume = 125.0000 cubic units; TSA = 150.0000 square units; D = 7.0711 units	Pass
3	s = 10 units	Volume = 1000.0000 cubic units; TSA = 600.0000 square units;	Volume = 1000.0000 cubic units; TSA = 600.0000 square units;	Pass

		D = 14.1421 units	D = 14.1421 units	
--	--	-------------------	-------------------	--

QUESTION 2 – Distance and Midpoint

Write a C program to find the distance and midpoint of a line segment when the coordinates of its two endpoints are given.

1. Problem Understanding

- **Input:** Coordinates (x_1, y_1) and (x_2, y_2) of the two endpoints.
- **Output:** Distance between the endpoints and the midpoint of the line segment.
- **Formula:** Distance = $\sqrt{[(x_2-x_1)^2 + (y_2-y_1)^2]}$; Midpoint = $((x_1+x_2)/2, (y_1+y_2)/2)$.

Formula Summary

Quantity	Formula
Distance	$d = \sqrt{[(x_2-x_1)^2 + (y_2-y_1)^2]}$
Midpoint	$M = ((x_1+x_2)/2, (y_1+y_2)/2)$

2. Standard Test Cases

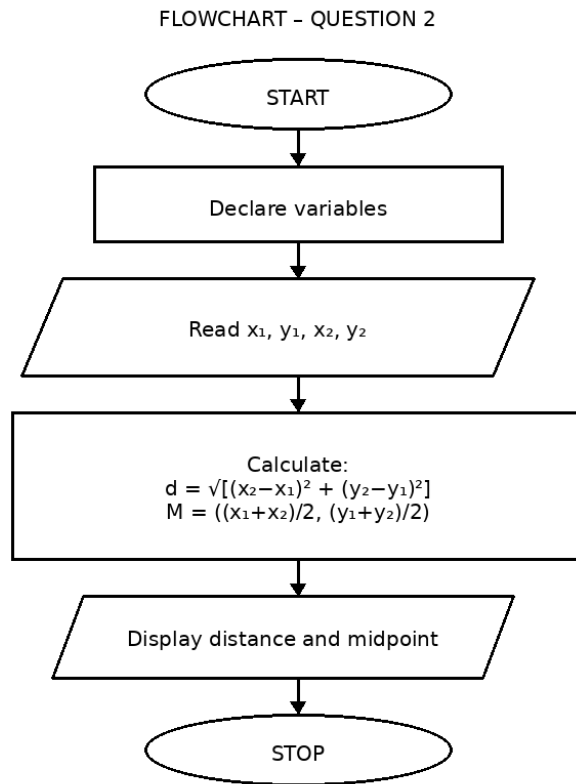
Test Case	Input (s)	Expected Output (s)	Actual Output (s)	Result (Pass/Fail)
1	$(x_1, y_1) = (0, 0);$ $(x_2, y_2) = (3, 4)$	Distance = 5.0000 units; Midpoint = (1.5000, 2.0000)	Distance = 5.0000 units; Midpoint = (1.5000, 2.0000)	Pass
2	$(x_1, y_1) = (1, 2);$ $(x_2, y_2) = (5, 5)$	Distance = 5.0000 units; Midpoint = (3.0000, 3.5000)	Distance = 5.0000 units; Midpoint = (3.0000, 3.5000)	Pass
3	$(x_1, y_1) = (-2, -3);$ $(x_2, y_2) = (4, 5)$	Distance = 10.0000 units; Midpoint = (1.0000, 1.0000)	Distance = 10.0000 units; Midpoint = (1.0000, 1.0000)	Pass

3. Algorithm

1. Start the program.
2. Declare variables for the coordinates, distance, and midpoint.
3. Read $x_1, y_1, x_2,$ and y_2 .
4. Calculate the distance using the distance formula.

5. Calculate the midpoint using the midpoint formula.
6. Display the distance and midpoint to four decimal places.
7. Stop the program.

4. Flowchart



5. C Program

```
#include <stdio.h>

#include <math.h>

int main(void)
{
    double x1, y1, x2, y2;

    double distance, mid_x, mid_y;

    printf("Enter x1 and y1: ");

    scanf("%lf %lf", &x1, &y1);

    printf("Enter x2 and y2: ");

    scanf("%lf %lf", &x2, &y2);
```

```

distance = sqrt((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1));

mid_x = (x1 + x2) / 2.0;

mid_y = (y1 + y2) / 2.0;

printf("Distance = %.4f units\\n", distance);

printf("Midpoint = (%.4f, %.4f)\\n", mid_x, mid_y);

return 0;

}

```

6. Compilation and Execution Evidence

```

kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)-[~/Desktop]
└─$ gcc question2.c -o question2 -lm

(kali@kali)-[~/Desktop]
└─$ ./question2
Enter x1 and y1: 0 0
Enter x2 and y2: 3 4
Distance = 5.0000 units\nMidpoint = (1.5000, 2.0000)\n

(kali@kali)-[~/Desktop]
└─$ ./question2
Enter x1 and y1: 1 2
Enter x2 and y2: 5 5
Distance = 5.0000 units\nMidpoint = (3.0000, 3.5000)\n

(kali@kali)-[~/Desktop]
└─$ ./question2
Enter x1 and y1: -2 -3
Enter x2 and y2: 4 5
Distance = 10.0000 units\nMidpoint = (1.0000, 1.0000)\n

(kali@kali)-[~/Desktop]
└─$ ss

```

7. Verification of All Test Cases

Test Case	Input (s)	Expected Output (s)	Actual Output (s)	Result (Pass/Fail)
1	(x ₁ ,y ₁) = (0,0); (x ₂ ,y ₂) = (3,4)	Distance = 5.0000 units; Midpoint = (1.5000, 2.0000)	Distance = 5.0000 units; Midpoint = (1.5000, 2.0000)	Pass
2	(x ₁ ,y ₁) = (1,2); (x ₂ ,y ₂) = (5,5)	Distance = 5.0000 units; Midpoint = (3.0000, 3.5000)	Distance = 5.0000 units; Midpoint = (3.0000, 3.5000)	Pass

3	$(x_1, y_1) = (-2, -3);$ $(x_2, y_2) = (4, 5)$	Distance = 10.0000 units; Midpoint = (1.0000, 1.0000)	Distance = 10.0000 units; Midpoint = (1.0000, 1.0000)	Pass
---	---	--	--	------